商品展示的业务依据

首屏 1 秒、三层缓存与深分页反模式

gomall · 业务侧 deck

今日大纲

首屏 1 秒决定订单

读路径：Cache Aside 是怎么挡住 DB 的

写路径：双删的业务必要性

HTTP cache：把流量挡在源站之外

反例：product/list 全表 COUNT

业务码、客诉与监控口径

首屏 1 秒决定订单

这一节回答四个问题

- 商品详情页是首屏黄金 1 秒，缓存命中 vs 回源 DB 在业务上差多少。
- Cache Aside 读路径 + 双删写路径，为什么**业务必须接受**最终一致而不是强一致。
- HTTP cache (ETag + Cache-Control) 哪几个接口能挂、TTL 怎么定。
- `product/list` 跑 24.5 RPS / p95 2.5s 是怎么从业务侧解读为反例的。

对应代码 *routes/router.go:39–45* / *middleware/httpcache.go* / *repository/cache/product.go* / *service/product.go*

首屏延迟与转化率：业务侧的硬性约束

首屏延迟	跳出率上升	业内常见处理
$\leq$ 1s	基线	不动
1 – 3s	+30%	进入优化排期
3 – 5s	+90%	列入 P0 故障复盘
$\geq$ 5s	用户基本走完	直接告警呼叫

Akamai 和 Google 在 2017 / 2018 公开的转化率曲线，是后端做缓存预算的业务依据。

gomall 的 `product/show` 压测 p95 = 3.01ms，留出 300 倍余量给业务侧的图片加载、首屏渲染。*stressTest/REPORT.md § 各链路压测结果（行 product/show）*

命中缓存 vs 回源 DB 的业务收益

路径	单次耗时	956K 行表能承担 QPS	业务侧含义
浏览器 304	~ 30ms (RTT)	不打到源站	流量被 CDN/浏览器卸掉
Redis 命中	~ 1ms	数万级	单机 Redis 上限
PK 回源 DB	3.01ms (p95)	6 万级	与 ping 同量级
全表扫	2.5s (p95)	24.5	直接告警

回源”不可怕，”扫表”才可怕。gomall `/product/show` 走主键 + 缓存兜底，业务侧从来不是瓶颈；`/product/list` 全表 COUNT 才是要修的那个。*stressTest/REPORT.md §1 基线 vs 业务接口；§3 商品列表反模式*

读路径：Cache Aside 是怎么挡住 DB 的

Cache Aside 读路径

- 读：先读缓存；缺则回源，再写回缓存。
- gomall 实现里多一道 `SETNX` 回源锁：缓存击穿时 **只放一个请求**打 DB，其它请求等 50ms 再读一次。
- 业务结果：商品详情压测 62K RPS，与 `/ping` 基线几乎重合。

service/product.go:42–68 ProductShow 三段式

读路径关键代码

Listing 1: *service/product.go:42–68 ProductShow*

```go
cached := &types.ProductResp{}
if err := cache.GetProductDetail(ctx, req.ID, cached); err == nil {
    return cached, nil // L1: 命中, ~1ms
}
locked, _ := cache.TryProductLock(ctx, req.ID) // L2: SETNX 抢回源权
if !locked {
    time.Sleep(50 * time.Millisecond) // L2': 让锁持有者回填
    if err := cache.GetProductDetail(ctx, req.ID, cached); err == nil {
        return cached, nil
    }
} else { defer cache.UnlockProduct(ctx, req.ID) }
pResp, err := s.loadProductFromDB(ctx, req.ID) // L3: 锁持有者打 DB
if err != nil { return nil, err }
_ = cache.SetProductDetail(ctx, req.ID, pResp) // L4: 回填, TTL=10min
return pResp, nil
```

读路径四步的业务含义

L1 命中 99% 的请求走这里。Redis ~ 1ms，对业务侧等价于” 没有数据库”。

L2 SETNX 缓存失效/被双删的瞬间，防止”100 个并发同时打 DB” 导致雪崩，业务侧叫**缓存击穿**。

L2’ 等 没抢到锁的请求 sleep 50ms 再读一次。读到就走 L1；读不到再硬回源（兜底，不能因为锁丢请求）。

L4 回填 TTL = `ProductDetailTTL = 10min`，这是 Redis 层；HTTP 层是 60s。两层 TTL 不一样，下一节解释。

*repository/cache/product.go:14 ProductDetailTTL = 10*time.Minute*

写路径：双删的业务必要性

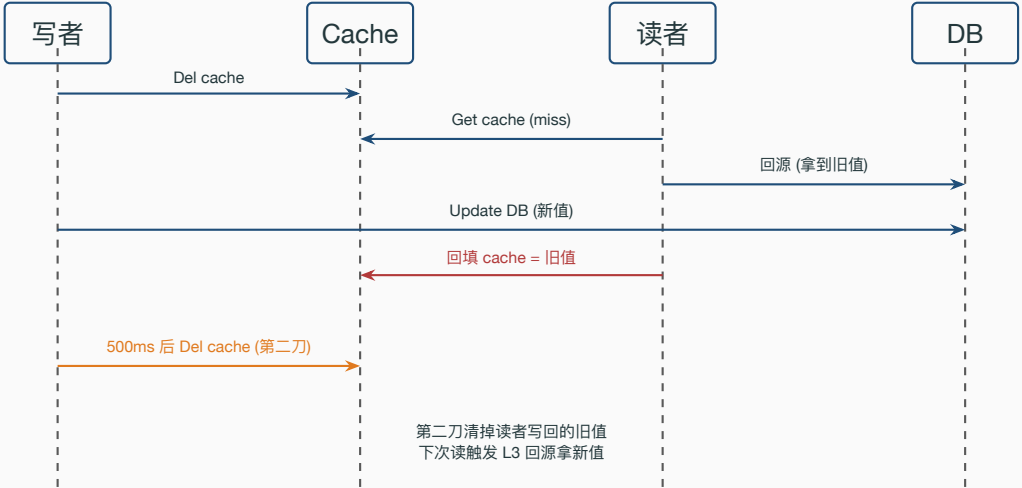
为什么不能” 写库 + 写缓存”

- 直觉做法：商家改价 → `UPDATE product` → `SET cache`。
- 业务现实里两个并发请求会：
  1. T1 改价 100 → 99
  2. T2 改价 100 → 88
  3. T2 先到 DB（写 88），T1 后到 DB（覆盖为 99）
  4. T1 后写 cache (99)，T2 先写 cache (88)
  5. DB = 99，Cache = 88 ——**永久不一致**
- 客诉版本：” 为什么列表写 88，详情写 99？ ”

> 业务侧不能依赖” 写缓存”，只能依赖” 删缓存”。

典型双写不一致问题；详见 *Facebook TAO* 论文 *§4*

延迟双删：第二刀挡哪个窗口



repository/cache/product.go:64–74 DoubleDeleteAsync (interval=500ms)

写路径关键代码：删-写-延迟删

Listing 2: *service/product.go:271–292 ProductUpdate*

```go
product := &model.Product{ /* 价格/标题等字段 */ }
_ = cache.DelProductDetail(ctx, req.ID) // 第一刀: 写库前先删
err = dao.NewProductDao(ctx).UpdateProduct(req.ID, product)
if err != nil {
    log.LogrusObj.Error(err); return
}
cache.DoubleDeleteAsync(req.ID, 0) // 第二刀: 500ms 后再删
emitProductChanged(ctx, req.ID, "update") // outbox -> RMQ -> ES
```

业务侧约定：商家改价后**最长 500ms + Redis TTL** 才能保证不出旧值；详情页 TTL=10min 时窗口实际由双删的 500ms 兜住。

业务侧为什么接受”500ms 旧值”

- 强一致方案（2PC / 同步写）单次写要 50–200ms，且任何下游慢都会让” 改价” 按钮卡住。
- 客服话术：” 改价后 1 秒内可能仍显示老价格，刷新即可。”——这是和产品/运营事先谈好的口径，不是 bug。
- 反过来，如果业务允许 1s 误差，第二刀的 500ms 已经是过设计；改成 200ms 也行。
- 真正不能容忍旧价的场景是**下单页**：那里 `/orders/create` 必须从 DB 现读价格，不走 cache。

> Cache Aside 是业务 SLA ” 最终一致 + $\leq$ 1s 窗口” 的工程实现，不是教科书最优解。

service/product.go:271–292; repository/cache/product.go:64–74

HTTP cache：把流量挡在源站之外

三层缓存漏斗

浏览器 cache (Cache-Control max-age) — >60% 流量在此截断

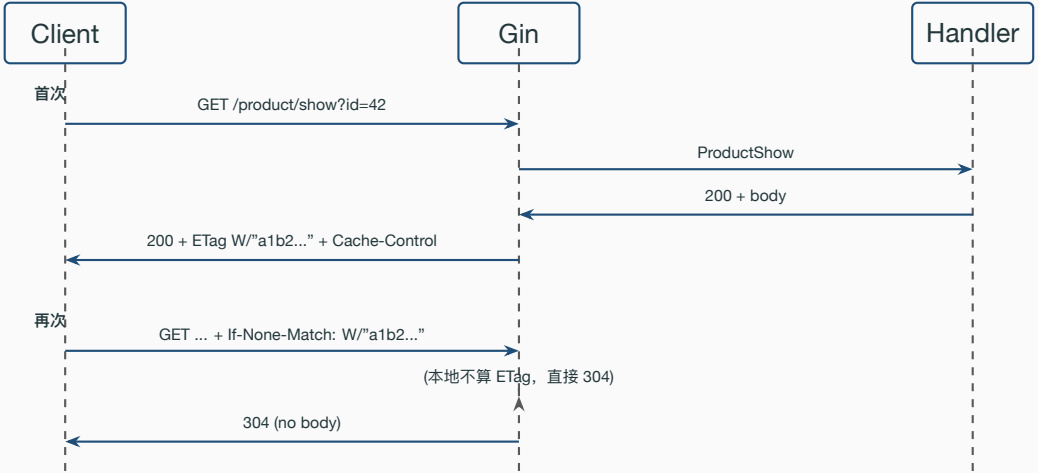
CDN / 反向代理 (ETag + If-None-Match) — 再截 15–20%

Redis (product:detail:{id}) — 再截 99% 剩余

MySQL (product PK) — 兜底回源

业务侧的数字：上一档 cache 卸下的流量，比下一档便宜一个数量级。浏览器 304 没有源站成本，CDN 一次回源覆盖一千个用户，Redis 一次回填覆盖 10min 内所有同商品请求。*routes/router.go:39–45* 三档挂载点

ETag 304 流程



middleware/httpcache.go:53–99 ETag 比对与 304 分支

ETag 中间件关键代码

Listing 3: *middleware/httpcache.go:78–99 比对 + 304*

```go
etag := weakETag(buf.body.Bytes()) // SHA-256 前 16 字节 -> W/"..."

if c.Request.Header.Get("If-None-Match") == etag {
    h := original.Header()
    h.Set("ETag", etag); h.Set("Cache-Control", cacheControl)
    h.Del("Content-Length"); h.Del("Content-Type")
    original.WriteHeader(http.StatusNotModified) // 304: 不写 body
    return
}
h := original.Header()
h.Set("ETag", etag); h.Set("Cache-Control", cacheControl)
h.Set("Content-Length", strconv.Itoa(buf.body.Len()))
original.WriteHeader(http.StatusOK)
_, _ = original.Write(buf.body.Bytes()) // 200: 正常回写
```

ETag 中间件逐行讲解

- **弱 ETag (`W/"..."`)**：只承诺” 语义等价”，不承诺字节级等价。压缩/分块都不影响命中，业务上更适合 JSON。
- **先 `c.Next()` 再比对**：必须等 handler 算完才能拿到 body，因此中间件用 `cacheBuffer` 包了 `ResponseWriter`。代价是**多一份内存**，但只对 GET 生效，不影响下单/支付。
- **只对 `status=200` 生效**：4xx / 5xx 直接透传，业务码 70001 (限流) 不会被错误地长期缓存。
- **命中 304 后删 `Content-Length` / `Content-Type`**：标准要求，否则 nginx / CDN 上游可能把空 body 算成 0 字节但仍发头，浏览器会拒收。

middleware/httpcache.go:47–99

TTL 怎么定：业务节奏决定

接口	max-age	业务理由	风险
`/product/show`	**60s**	单品价格/库存可能改	改价 1min 内显示老价
`/product/list`	30s	列表翻页频繁，改卖家少	上架 30s 内不可见
`/category/list`	**300s**	类目几乎不变	几乎无
`/carousels`	300s	运营每天换一次轮播图	几乎无

数字不是抄的，是和**产品同学**对齐过的” 用户能容忍多久看到旧值”。

60s 是为了和 ProductDetailTTL (10min) 解耦：即使 Redis 还是旧值，HTTP 层 60s 后会强制重新协商一次。*routes/router.go:39 (30s) / :40 (60s) / :44 (300s) / :45 (300s)*

为什么 POST/PUT 接口不能挂

- `HTTPCache` 中间件首行就 `if method != GET && method != HEAD: return`。
- 业务原因：POST 是” 发动作”，挂 cache 意味着**重复触发同一动作**，跟幂等机制冲突。
- 例如 `POST /orders/create` 如果命中浏览器 cache 会让用户以为下单成功，实际**没打到后端**，对账会少一笔订单。
- gomall 的写接口（create/update/delete/paydown）都走幂等中间件 + DB 兜底，从不进入 HTTP cache 层。

middleware/httpcache.go:54–58 方法判断

反例：product/list 全表 COUNT

反例数字：24.5 RPS / p95 2.5s

接口	实测 RPS	p95
`/ping` 基线	64,254	3.51ms
`/product/show`	62,226	3.01ms
`/product/list` (反例)	**24.5**	**2.50s**

同样是商品域的 GET 接口，吞吐差 **2,500 倍**，p95 差 **800 倍**。

区别只有一个：`ShowProduct` 走主键，`ListProducts` 多了一个 `SELECT COUNT(*)`。

stressTest/REPORT.md § 各链路压测结果（行 product/list）

扫表示意：1M 行的 COUNT 怎么炸的

```
SELECT COUNT(*) FROM product WHERE category_id=?
```

全表扫: 每次请求 ~ 2 s

50 VU * 2s → 24.5 RPS

product 表 956K 行 / 364 MB

- `Count(&total)` 在没有 `category_id` 索引时退化为全表扫。
- 即使有索引，InnoDB 的 `COUNT(*)` 不能像 MyISAM 那样常量返回，仍要走索引页。
- 业务侧问的从来不是” 商品一共多少件”，而是” 我这一页有多少件”——total 大多是**产品需求过度**。

repository/db/dao/product.go:58–64 CountProductByCondition

反例代码

Listing 4: *repository/db/dao/product.go:58–64*

```go
// CountProductByCondition 根据情况获取商品的数量
func (dao *ProductDao) CountProductByCondition(condition map[string]interface{}) (total int64,
     err error) {
   err = dao.DB.Model(&model.Product{}).
      Where(condition).Count(&total).Error // 1M 行表 + 无 category_id 索引 -> 全表扫
   return
}
```

service 调用点： *service/product.go:195–204 ListProducts 同时调 ListProductByCondition + CountProductByCondition*

三种修法对业务的代价

方案	代价	业务可见变化
建 `category_id` 索引	写入慢 5–10%	几乎无
近似计数 (`TABLE STATUS`)	total 有 5% 误差	显示” 约 1,000 件”
干掉 total，前端无限滚动	产品改交互	翻页器消失

后端常常想直接选” 建索引”，但和产品/运营对齐后会发现**无限滚动**才是真正合理的选择：用户分页到第 50 页的概率 $< 0.1\%$，业务上没人在乎 total。*stressTest/REPORT.md §3* 商品列表反模式改进方向

业务码、客诉与监控口径

业务码：60002 的真实含义

状态码	含义	客服话术
60001	幂等 token 无效	” 请刷新页面后重试”
60002	幂等正在处理	” 您的请求正在处理，1 秒后自动刷新”
70001	限流命中	” 活动太火爆，请稍后重试”
70002	熔断开启	” 支付服务暂忙，1 分钟后”

注意 60002 不是

cache 命中（HTTP 304 才是），而是**幂等命中**——同一 Idempotency-Key 在第一笔还没结束就再来一次。两者对客服来说都是” 无需重复操作”，话术接近；但监控口径不同：304 不写日志、不计错误率；60002 要计入” 幂等保护次数”。*pkg/e/code.go:53–58* 业务码定义

客诉拆解：” 为什么显示老价格”

场景 A 商家改价后 $\leq$ 500ms 内用户刷新。
原因：双删第二刀还没到。处理：让用户再刷一次。

场景 B 商家改价后 1 – 60s 内用户刷新。
原因：浏览器/CDN 强 cache（Cache-Control max-age=60）。处理：教用户硬刷新 (`Cmd+Shift+R`)，或商家急时让运营走” 强制下架重新上架”。

场景 C 商家改价后 $>$ 10min 仍是旧值。
原因：双删失败或 outbox 没投递。处理：开 P2 工单，查 `outbox` 表 `ProductChanged` 事件。

service/product.go:282–289 双删 + emitProductChanged 三联

业务侧的最终目标

- 首屏 1 秒：靠浏览器 304 + CDN 长缓存，源站只承担兜底。
- 命中率 > 99%：靠 Redis Cache Aside + SETNX 单飞回源。
- 改价 1s 内可见：靠延迟双删 + outbox 异步索引。
- 翻页不炸：靠下掉全表 COUNT 改无限滚动。

商品展示不是” 把数据从 DB 取出来”，是**把流量挡在 DB 之外**。

面试 Q&A

Q1：双删第二刀的间隔为什么是 500ms，不是 200ms 或 1s?

A：业务侧承诺” 改价 1s 内可见”。第一刀已经挡 99%，第二刀只为兜并发读 + 回写窗口；500ms 是估计的” 读者从 cache miss 到回填完” 的上界，再保守一倍也只到 1s。

Q2：`/product/show` 已经 62K RPS 了，为什么还要挂 HTTP cache?

A：62K RPS 是**单机源站极限**，不可水平扩。挂 HTTP cache 后浏览器 / CDN 卸掉 60–80% 流量，源站压力降到 1 万级，留出余量给写路径。

Q3：商家改价后客诉” 显示老价格”，怎么定位?

A：先看时间窗口。≤ 1s 是双删窗口（正常）；1 – 60s 是 HTTP cache（教硬刷新）；> 10min 是 outbox / 双删失败（开工单）。

面试 Q&A（续）

Q4：为什么 `product/list` 不挂 Redis cache 而 `product/show` 挂?

A：列表的 key 维度是 `(category, page, filter)` 笛卡尔积，cache 命中率低；单品按 id 命中率 → 99%。列表更适合 HTTP cache (URL 是天然 key) 或前端无限滚动。

Q5：弱 ETag 和强 ETag 怎么选?

A：JSON 接口选弱（`W/"..."`），允许压缩/格式变化；二进制资源选强。gomall 全部是 JSON，统一弱 ETag。

Q6：商家秒级改价、用户必须秒级看到，怎么办?

A：改业务约定，不改技术。要么 max-age=0 取消强缓存（仅 ETag 协商），要么对该商品走**下单页直读 DB** 的兜底路径。技术上” 既要强缓存又要强一致” 在分布式里没有解。

代码位置一览

读路径 Cache Aside *service/product.go:42–68 ProductShow*

写路径双删 *service/product.go:271–292 ProductUpdate*

Cache 包装 *repository/cache/product.go:30–74 Get/Set/Del/DoubleDeleteAsync*

SETNX 回源锁 *repository/cache/product.go:55–62 TryProductLock*

HTTP cache 中间件 *middleware/httpcache.go:47–106 HTTPCache + weakETag*

路由挂载（show 60s / list 30s） *routes/router.go:39–40*

路由挂载（category / carousels 300s） *routes/router.go:44–45*

反例：全表 COUNT *repository/db/dao/product.go:58–64*

业务码 *pkg/e/code.go:53–58 (60001/60002/70001/70002)*

压测数字 *stressTest/REPORT.md* 链路压测表 + *§3* 反模式